

Modül 18: Eşzamanlılık Temelleri

İş parçacıkları veri paylaştığında iki şey ters gider, ve bunlar aynı şey değildir.

Yarışlar araya girmeyele ilgilidir: iki iş parçacığı bir işlemin ortasında birbirine karışır. **Görünürlük**, bir iş parçacığının diğerinin yazdığını hiç görmemesiyle ilgilidir. Bu modül ikisini de gösterir, sonra her birini çözen araçları anlatır.

Neler Öğreneceksiniz

- İş parçacıkları, kesme, ve neden kendiniz iş parçacığı oluşturmamalısınız
- `count++` neden artışları kaybediyor, yüz binlerce kayıpla gösterilerek
- `synchronized`, atomikler ve `LongAdder`, ve her birinin bedeli
- Bir döngünün hiç bitmediği görünürlük problemi
- `ExecutorService`, ve sessiz kalan başarısızlık biçimi

Başka Bir Dilden Geliyorsanız

	Python	Java
Gerçek paralellik	GIL engelliyor	evet, gerçekten
<code>x += 1</code> üzerinde yarış	mümkün	ölçekte kesin
Kilit	<code>threading.Lock</code>	<code>synchronized</code> , <code>ReentrantLock</code>
Atomik sayaç	yerleşik yok	<code>AtomicInteger</code>
İş parçacığı havuzu	<code>ThreadPoolExecutor</code>	<code>ExecutorService</code>
Görünürlük anahtar kelimesi	gerekmiyor	<code>volatile</code>

Python'un global yorumlayıcı kilidi bunların çoğunu gizler. Java iş parçacıklarını gerçek çekirdeklerde paralel çalıştırır, dolayısıyla buradaki her tehlike gerçekten karşılaşacağınız bir tehliktir.

Ders

Doğrudan iş parçacıkları

```
Thread worker = new Thread(() -> doWork(), "worker-1");
worker.start();
worker.join();
```

`start()` ile `run()` aynı şey değildir. `start()` bir iş parçacığı oluşturur; `run()` yalnızca mevcut olanda bir metod çağırır. Araya girecek bir şey olmadığında çıktı aynı olduğu için hata çalışıyormuş gibi görünür.

Kesme bir istektir, bir öldürme değil. `InterruptedException` yakalamak bayrağı temizler, dolayısıyla geri koyun:

```
catch (InterruptedException e) {  
    Thread.currentThread().interrupt();  
}
```

`Thread.stop()` 2000'de kullanımdan kaldırıldı ve artık tamamen kaldırıldı, çünkü bir iş parçacığını işlemin ortasında öldürmek, değiştirdiği her şeyi bilinmeyen bir durumda bırakır.

Neden kendiniz iş parçacığı oluşturmamalısınız: bir platform iş parçacığı yaklaşık 1MB yığın alanına mal olur, kaçının var olduğuna dair bir sınır yoktur, ve hiçbir şey sonuçları toplamaz ya da hataları yukarı iletmez.

Yarış

`TheRaceCondition.java` dosyasından, her biri 200.000 kez artıran dört iş parçacığı:

```
expected total: 800000  
actual total:   242657  
-> 557343 increments were LOST
```

`count++` üç işlemdir: oku, ekle, yaz. İki iş parçacığı da `101` yazmadan önce `100` okuyabilir, ve bir artış kaybolur.

Aynı biçim her yerde görünür: `x += 1`, `list.add(item)`, `balance -= amount`, ve her denetle-sonra-davran çifti:

```
if (!map.containsKey(k)) { map.put(k, v); }
```

İki iş parçacığı da biri koymadan önce denetimi geçebilir. `Map` sınıfının tek atomik işlem olarak `putIfAbsent` metoduna sahip olmasının sebebi budur.

Üç çözüm

`FixingTheRace.java` dosyasından:

unsafe ++	796131	WRONG
synchronized	800000	correct
AtomicInteger	800000	correct
LongAdder	800000	correct

synchronized karşılıklı dışlama ve bir önce-olur kenarı verir: bir iş parçacığının kilidi bırakmadan önce yaptığı her şey, kilidi sonra alan iş parçacığına görünür. **İnsanların unuttuğu ikinci garanti budur**, ve **synchronized** metodunun görünürlüğü de çözmesinin sebebi. Bedeli, bloke olan iş parçacıklarının sıraya girmesidir.

Atomikler bir karşılaştırmak-ve-değiştir komutu kullanır: yalnızca hiçbir şey değişmediyse geri yaz, aksi hâlde yeniden dene. Bloke olma yok, ama çekişme altında yeniden denemeler bedel olur.

LongAdder birkaç iç hücre tutar, dolayısıyla iş parçacıkları çoğunlukla aynı belleğe dokunmaz. Yoğun yazma çekişmesi altında **AtomicLong** sınıfından hızlı, okuması yavaştır. Metrikler için onu; her güncellemeden sonra değere ihtiyacınız olduğunda **AtomicLong** kullanın.

Ve genellikle en iyisi olan dördüncü seçenek: durumu hiç paylaşmayın. Modül 14 paralel stream'ler için aynı noktayı yapmıştı, orada çözüm her iş parçacığına kendi kabını veren **collect** idi. Yaşayamayacağınız bir yarış, senkronize ettiğiniz bir yarıştan ucuzdur.

Neye kilitlememeli:

Yapmayın	Çünkü
<code>synchronized (this)</code>	referansı olan herkes sizi kilitleyebilir
<code>synchronized (SomeClass.class)</code>	aynısı, ama küresel olarak
bir <code>String</code> sabiti	sabitler havuzlanır ve ilgisiz kodla paylaşılır
kutulanmış bir <code>Integer</code>	128 altındaki değerler önbelleklenir

Bir `private final Object` kullanın.

Görünürlük

İnsanların beklemediği kısım bu. `MemoryVisibility.java` dosyasında bir iş parçacığı bir bayrak üzerinde dönüyor ve bir diğeri onu ayarlıyor:

```
main set plainFlag = true
after waiting 1.5s, reader still running: true

main set volatileFlag = true
after waiting 1.5s, reader still running: false
```

Düz okuyucu yazmayı **hiç görmedi**. Burada hiç araya girme yok.

O döngüdeki hiçbir şey paylaşılan duruma dokunmuyor, dolayısıyla JIT `plainFlag` değerinin değişmeyeceğini varsayıp okumayı tamamen döngü dışına çıkarabilir ve onu `while (true)` hâline getirebilir. Bu **yasal** bir optimizasyondur. Java Bellek Modeli, bir iş parçacığının diğerinin yazdığını görmesini yalnızca ikisini bir önce-olur ilişkisi bağladığında söz verir, ve düz bir alan böyle bir ilişki kurmaz.

volatile ne yapar: her okuma ana belleğe gider, ve önce-olur kurar, dolayısıyla bir volatile yazmadan önce yazılan her şey eşleşen okumadan sonra görünür.

Ne yapmaz: `count++` işlemini atomik kılmaz. Üç işlem üç işlem olarak kalır.

volatile bayraklar ve bir referans yayımlamak içindir. **synchronized** , atomikler ve kilitler bileşik eylemler içindir.

ExecutorService

```
try (ExecutorService pool = Executors.newFixedThreadPool(3)) {  
    Future<Integer> result = pool.submit(() -> compute());  
    System.out.println(result.get());  
}
```

ExecutorService Java 19'dan beri `AutoCloseable` , dolayısıyla try-with-resources insanların düzenli olarak yanlış yaptığı `shutdown` ve `awaitTermination` çiftinin yerini alıyor.

Gerçek bir tuzak, `UsingExecutors.java` dosyasından:

```
submitted, and nothing has been reported yet  
get() threw ExecutionException  
caused by: java.lang.IllegalStateException: the task failed
```

`submit()` ile gönderilen, fırlatan ve `Future` nesnesi hiç incelenmeyen bir görev **sessizce başarısız olur**. Hiçbir şey loglanmaz, hiçbir şey çıkmaz. Varsayılan işleyicinin raporlamasını istiyorsanız `execute()` kullanın, ya da `Future` nesnesini her zaman inceleyin.

Havuz	Kullanım
<code>newFixedThreadPool(n)</code>	sınırlı; CPU işi için güvenli varsayılan
<code>newCachedThreadPool()</code>	sınırsız; görev patlaması iş parçacığı patlaması yapar
<code>newSingleThreadExecutor()</code>	teker teker, sırayla
<code>newScheduledThreadPool(n)</code>	gecikmeli ve tekrarlayan

IO ağırlıklı iş için bunların hepsi çok az iş parçacığı ile çok fazlası arasında bir seçim dayatır. Modül 19 bu seçimi ortadan kaldırıyor.

Çalıştırın

```
java modules/18-concurrency-basics/examples/ThreadsAndRunnables.java
java modules/18-concurrency-basics/examples/TheRaceCondition.java
java modules/18-concurrency-basics/examples/FixingTheRace.java
java modules/18-concurrency-basics/examples/MemoryVisibility.java
java modules/18-concurrency-basics/examples/UsingExecutors.java

./scripts/verify-examples.sh modules/18-concurrency-basics
```

Yarış örneğini birkaç kez çalıştırın. Kaybedilen sayı her seferinde değişir, ve çok çekirdekli bir makinede asla sıfır olmaz. Ders o tutarsızlıktır.

Sık Yapılan Hatalar

`start()` yerine `run()` çağırmak. Hiçbir iş parçacığı oluşturulmaz.

`volatile` kelimesinin `++` işlemini güvenli kıldığını varsaymak. Kılmaz.

`InterruptedException` yutmak. Bayrağı geri koyun, yoksa kapanma sinyali kaybolur.

`this`, bir sınıf, bir `String` ya da kutulanmış bir `Integer` üzerine kilitlemek. Hepsi düşündüğünüzden çok daha geniş paylaşılır.

Bir `Future` nesnesini denetlememek. İstisnalar tamamen kaybolur.

Bir sunucuda `newCachedThreadPool`. Yük altında sınırsız iş parçacığı oluşturma.

Eşzamanlı bir haritada denetle-sonra-davran. `containsKey` sonra `put` iki işlemdir. `putIfAbsent` ya da `compute` kullanın.

Güvende olmak için her şeyi senkronize etmek. Çekişme, paralel sürümü ardışık olandan yavaş yapabilir, ve Modül 14 tam olarak bunu göstermişti.

Özet Çıkarımlar

- `count++` üç işlemdir, dolayısıyla çekişme altında artış kaybeder.
- `synchronized` karşılıklı dışlama ve görünürlük verir. Atomikler kilitsiz güncelleme verir. `LongAdder` yoğun yazma çekişmesinde kazanır.
- Görünürlük ayrı bir tehlikedir. Düz bir alan önbelleklenebilir ya da döngü dışına çıkarılabilir, ve okuyucu yazmayı hiç görmeyebilir.
- `volatile` görünürlüğü çözer, atomikliği değil.
- Duruma erişimi senkronize etmek yerine paylaşmamayı tercih edin.
- Bir `ExecutorService` kullanın, try-with-resources ile kapatın, ve `Future` nesnesini her zaman inceleyin.

Ödev

[homework/README.md](#)

Bir sayacı dört biçimde kurun, kırın, sonra bir denetle-sonra-davran yarışını düzeltin. Referans çözüm [solutions/18-concurrency-basics/](#) altında.